

Software Engineering

Software Development Processes

Software Development Processes: Software Engineering Activities

What is “Software Engineering”?

Development Processes
Quality Management
Requirements Engineering
System Design
Architecture
Implementation
Testing
Delivery
Maintenance
Scaling
Operations

Software Engineering Activities

When engineering Software, teams repeatedly engage in the following activities

Discovery

Requirements Engineering

Design

Implementation

Testing

Delivery

Validation

Operations

Maintenance

Which problem do we want to solve?

How can we solve this problem?

How does our solution look like?

How do we realise our solution?

Are we correctly solving the problem?

How do we bring the solution to the users?

Does the solution solve the right problem?

How do we ensure the availability of the solution?

How do we ensure that our solution doesn't degrade over time?

Software Engineering: Basic Terminology

Roles

Stakeholder

Project Manager / Product Owner / Product Manager

(UI/UX) Designer

Developer / Software Engineer

Tester / Quality (Assurance) Manager / Quality Engineer

Operations Admin / Infrastructure Engineer / Site Reliability Engineer / ~~The DevOps guy~~

User

Software Engineering Activities

	Product Owner	Designer	Software Engineer	Quality Engineer	Infrastructure Engineer
Discovery					
Requirements Eng.					()
Design					
Implementation					
Testing			 / 	 / 	
Delivery					
Validation					
Operations					
Maintenance					

Software Development Processes: Sequential Models

What is a Software Development Process?

The development of a software system through the execution of activities within a specific order

- Helps to gain a common understanding of the general procedures
- Specifies how activities are ordered and modelled
- Establishes way points for monitoring and controlling

Software Development Processes

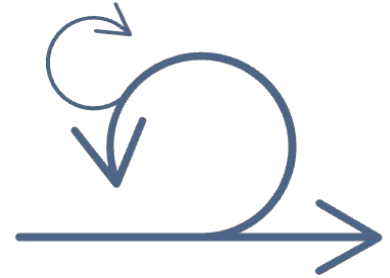
Sequential – usage in high risk and military applications

- Plan upfront for a long term (months, years)
- Follow rules and move on sequentially
- Avoid deviations from the original plan

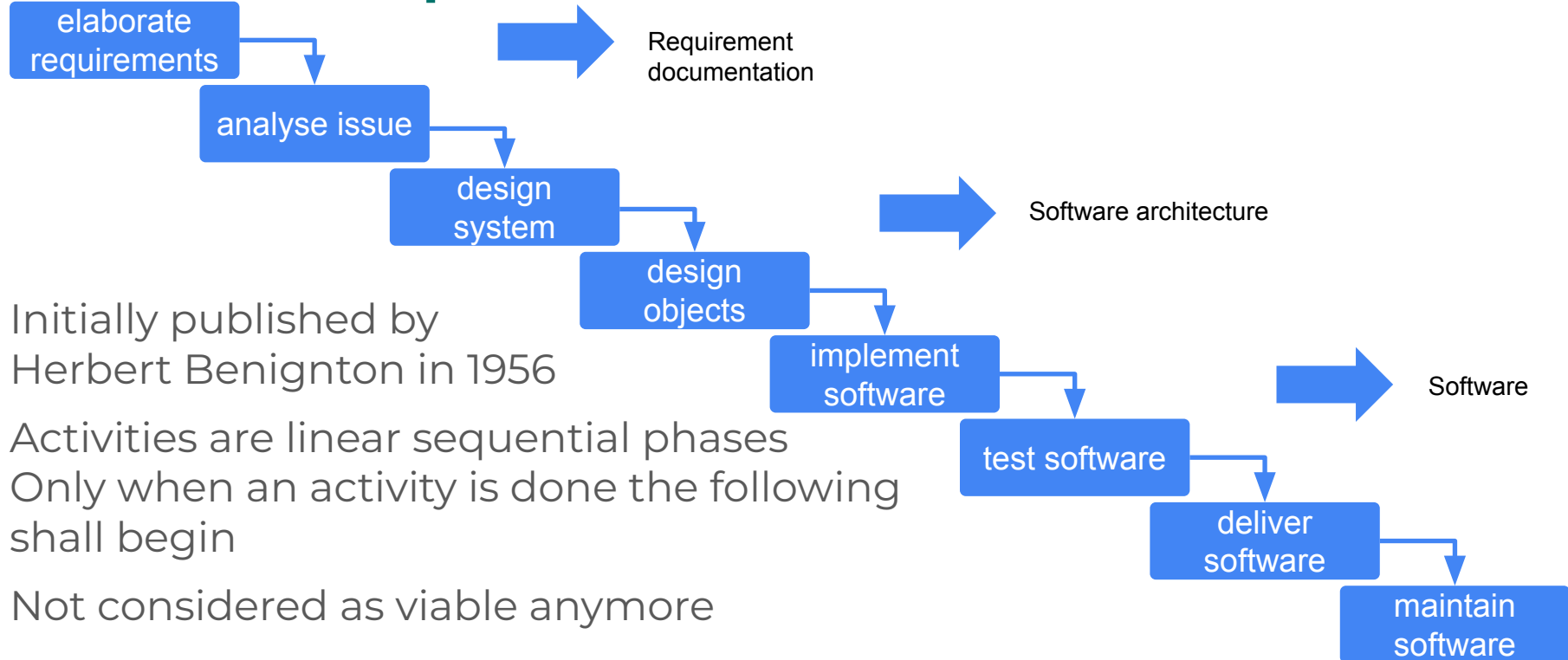


Iterative – used in almost all tech organisations today

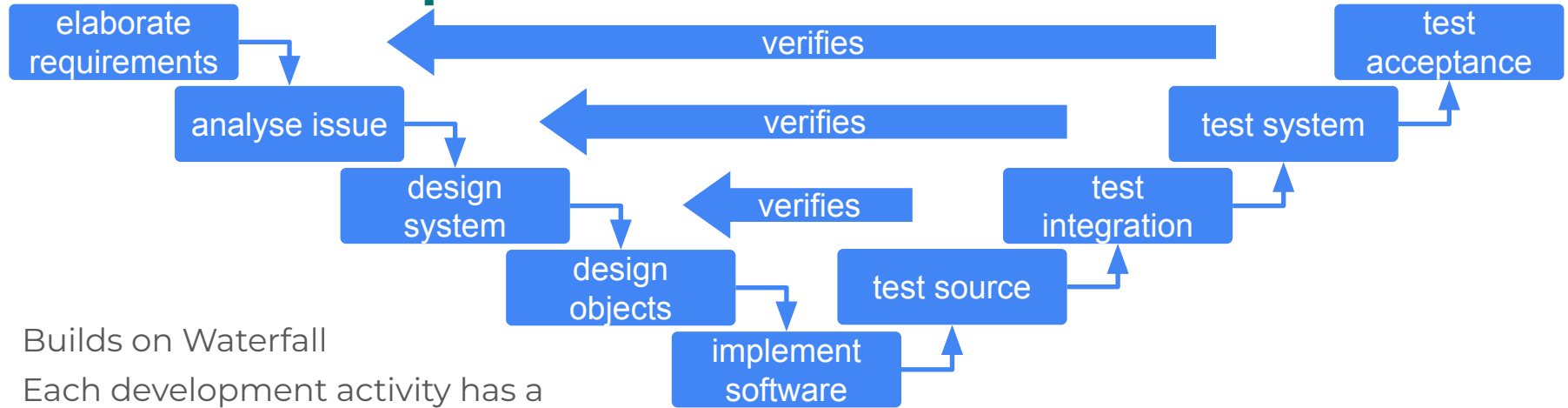
- Only plans as much as needed
- Executes small iterations (hours, days, weeks)
- Embraces change as inevitable, continuously inspect and adapt
- Agile ways of working use iterative processes



Waterfall (Sequential)



V-Modell (Sequential)



- Builds on Waterfall
- Each development activity has a corresponding testing activity
- When a testing activity fails the process must move back to the corresponding development activity and repeat the phases. Still linear and sequential
- Still considered viable in industrial and military applications (sometimes even must-have)
- Parts of this approach are used in hybrid models

Software Development Processes: Iterative Models

CODE

Lorem ipsum, or lipsum as it is sometimes Lorem

PLAN

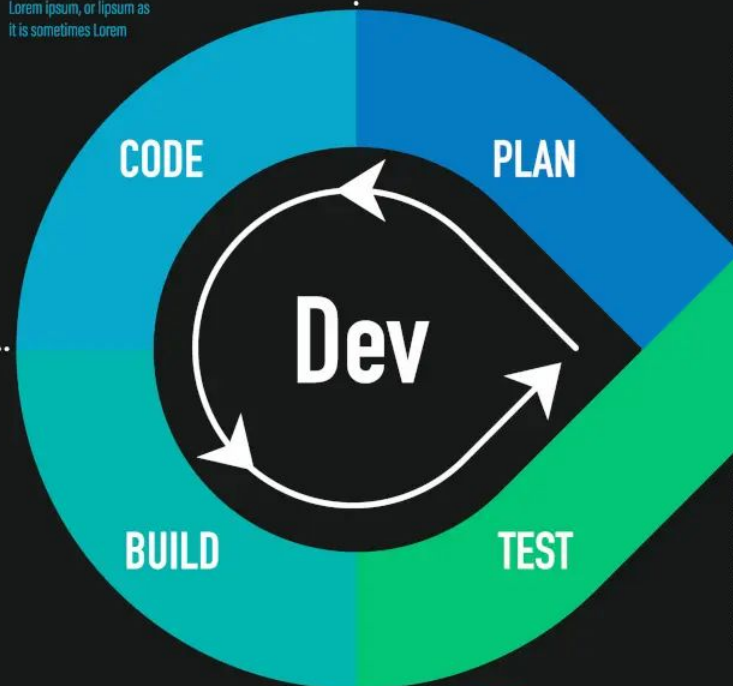
Lorem ipsum, or lipsum as it is sometimes Lorem

RELEASE

Lorem ipsum, or lipsum as it is sometimes Lorem

DEPLOY

Lorem ipsum, or lipsum as it is sometimes Lorem



RELEASE

DEPLOY

MONITOR

OPERATE

Ops

BUILD

Lorem ipsum, or lipsum as it is sometimes Lorem

TEST

Lorem ipsum, or lipsum as it is sometimes Lorem

MONITOR

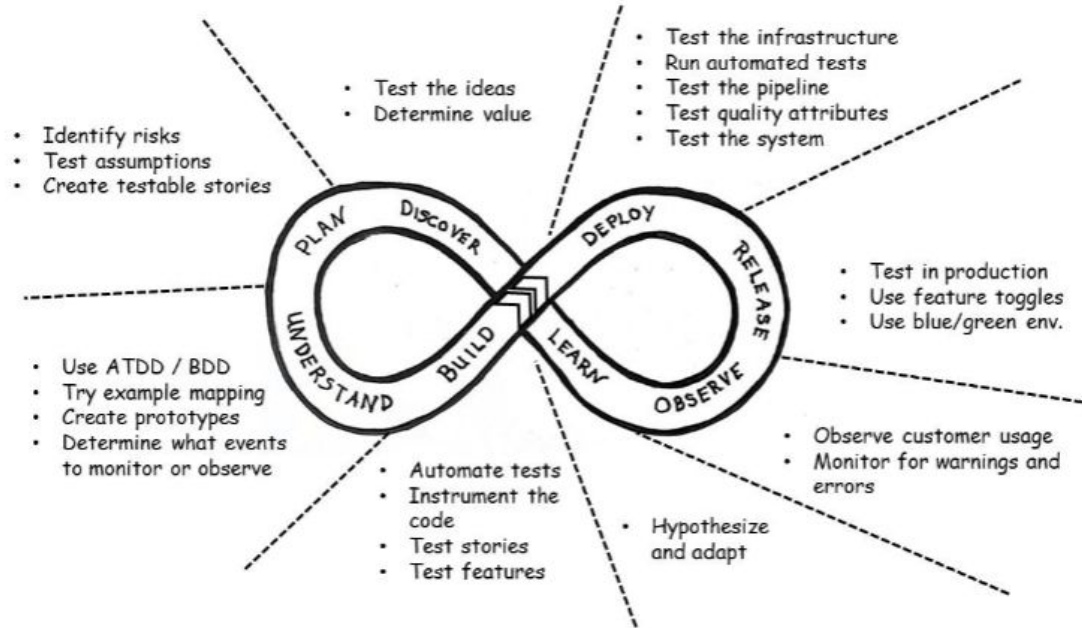
Lorem ipsum, or lipsum as it is sometimes Lorem

OPERATE

Lorem ipsum, or lipsum as it is sometimes Lorem



Software Testing Models: Holistic Testing



Source: [Holistic Testing Mini Book by Janet Gregory and Lisa Crispin](#), model was most likely created in the late 2010s

What does working agile mean?

The roots of agile ways of working

The Agile Manifesto

Individuals and interactions over processes and tools

Working software over comprehensive documentation

Customer collaboration over contract negotiation

Responding to change over following a plan

Further reading: <https://agilemanifesto.org/>

The roots of agile ways of working

The Agile Manifesto

signed in 2001

- Kent Beck, Mike Beedle, Arie van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Ron Jeffries, Jon Kern, Brian Marick, Robert C. Martin, Steve Mellor, Ken Schwaber, Jeff Sutherland, Dave Thomas

The roots of agile ways of working

XP - Extreme Programming

- emerged in the 90ies in collaboration
 - Don Wells, Kent Beck, Ron Jeffries, Martin Fowler, Chet Hendrikson, Ward Cunningham, Angelique Thouvenin-Martin, Laurie Williams
- website by Don Wells published in 1999
- book “XP explained” by Kent Beck published in 1999

Sources and further reading:

- <http://www.extremeprogramming.org/>
- https://en.wikipedia.org/wiki/Extreme_programming

The roots of agile ways of working

Scrum

- first use of the term “Scrum” in an article about the changes in Product Development processes by Takeuchi & Nonaka 1986
- emerged in the 90ies in collaboration
 - Ken Schwaber, Jeff Sutherland, John Scumniotales, Jeff McKenna, Babatunde Ogunnaike
- Scrum Alliance was founded in 2002
- first Scrum Guide published in 2009

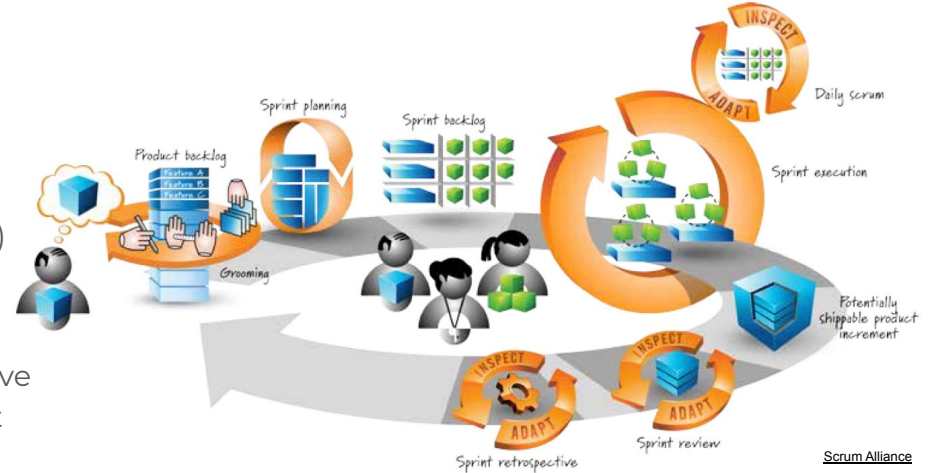
Sources and further reading:

- <https://www.scrumalliance.org/>
- <https://scrumguides.org/>
- [https://en.wikipedia.org/wiki/Scrum_\(software_development\)](https://en.wikipedia.org/wiki/Scrum_(software_development))

Software Development Processes: Scrum (Iterative, Agile)

Scrum

- Defines roles
 - Product Owner (stakeholder representative)
 - Development Team (3 – 9 members, cross-functional)
 - Scrum Master (facilitator of the process)
- Defines activities and artifacts
 - refinement, sprint, daily, planning, review, retrospective
 - user stories, product backlog, sprint backlog, product increment
- Defines process
 - user stories are pulled for sprint from product backlog by team
 - team estimates effort in story points and commits to sprint
 - sprints have fixed scope and are timeboxed (2 – 4 weeks)
 - daily scrum enables team communication on a daily basis (15 min.)
 - each sprint shall deliver a shippable product increment that is reviewed by the product owner in the sprint review



**Scrum is heavily used since 2002.
Has 2 main certification bodies
and tons of adaptations.**

Software Development Processes: Kanban (Iterative, Agile)

What is flow? - Exercise 10 minutes

In groups of 2-3:

- Go to <https://traffic-simulation.de/>
 - (scan QR code)
- Play with the parameters, especially
 - Onramp flow
 - Max speed
- How are traffic jams created?



Software Development Processes: Kanban

Kanban's main goal is to ***optimise for flow***.

Software Development Processes:

Kanban

- Kanban as a Lean Manufacturing Practise was invented at Toyota in the 1950ies
- “To produce only what is needed, when it is needed and in the amount needed.” - Taiichi Ohno
 - “Just-in-time” production
- Identified the 7 types of waste
- Kanban strives to eliminate waste to optimise for flow (throughput)

Sources:

- <https://leanmh.com/what-taiichi-ohno-taught-us-about-lean>
- <https://kanbantool.com/kanban-guide/kanban-history>

Image source: [Wikimedia, Ohno-Taiichi](#)



Software Development Processes: Kanban

Utilisation

Utilisation is the ratio of the capacity that **someone or something is in use** to the total capacity that it could be used

- Traditional capacity planning aims to maximise utilisation of resources and people

Throughput

Throughput refers to the rate at which a process **creates output** within a given time period.

- Kanban and other lean methodologies aim at maximising throughput (aka flow)

Exercise 10 minutes - in groups of 2-3:
Find examples of processes that either aim to maximise utilisation or throughput

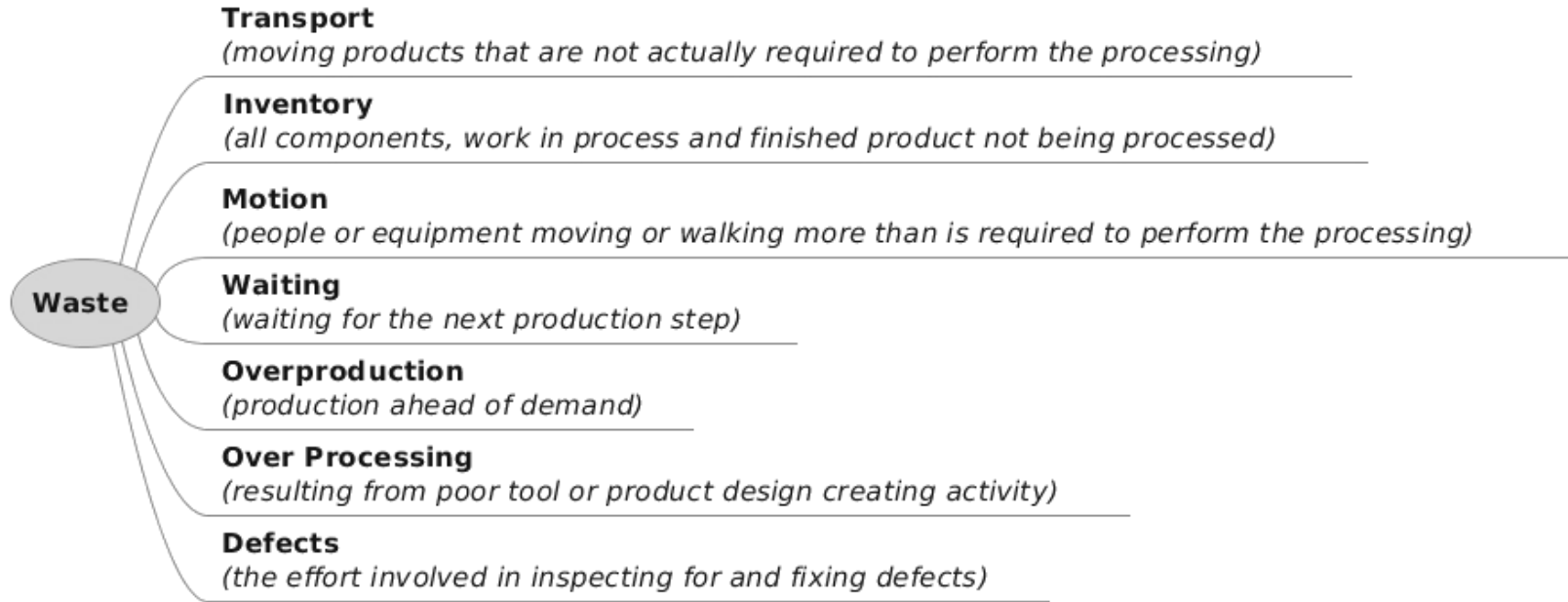
Software Development Processes: Kanban

What is **waste**?

“non-value-adding activities within a process”

Source: kanbantool.com

Software Development Processes: Kanban



Source: kanbantool.com

Software Development Processes: Kanban

Most important types of waste in Software Engineering

(leaving out transport and over-production here) - Source: kanbantool.com

Inventory

In knowledge work, the inventory waste shows as the exceeding of your Kanban board's WIP (work in progress) limit. The more tasks you start to work on at once, the more slowly they will get done. It's much more efficient to stick to the limit and not begin new work until your head is free from previously opened tasks.

Motion

For non-physical work, motion-related waste within a process stage are usually unnecessary emails, calls, meetings that could all be replaced with a status change on the individual task card.

Software Development Processes: Kanban

Most important types of waste in Software Engineering

(leaving out transport and over-production here) - Source: kanbantool.com

Waiting

In many ways, the waste caused by items waiting to be processed is the easiest to spot but not always equally as easy to remove. Jobs or tasks will be queuing for extended time between process steps when your process either lacks the ideal capacity to pull all tasks at once, or when the started items processing is not going optimally. Bottlenecks form, with consequences visible on all following process stages.

The standard Lean methods to reduce waiting are applying and adjusting WIP limits. Crucially, consider limiting the designated “waiting” steps - i.e., the “ready for another stage” ones. If you decide how many items can pile up in them before teams from other process stages step in to help, you will likely avoid late deliveries and costly impediments.

Software Development Processes: Kanban

Most important types of waste in Software Engineering

(leaving out transport and over-production here) - Source: kanbantool.com

Over-processing

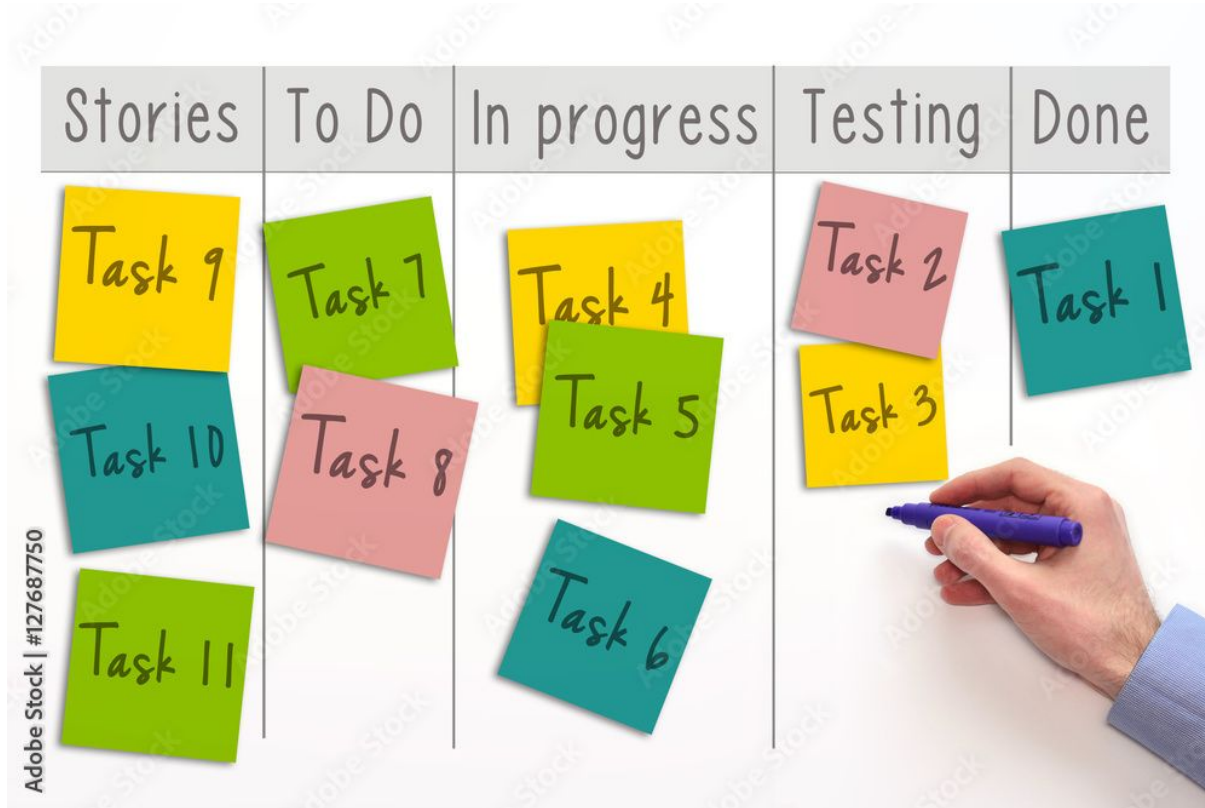
The problem with over-processing items usually ties to miscommunication. For that reason, it is vital that you name the exact state, quality and shape of the desired final product, and expect not less and no more than exactly that done. When unsure, agree to do less rather than more. This form of waste also applies to using equipment more sophisticated (so more expensive) than necessary to deliver a job. Similarly, it refers to putting overqualified personnel on simple jobs.

Defects

Defects in production are quite a standard form of waste. Not only do they demand a repeat in processing. When undetected in time, they will drive the customer satisfaction down and may cause a recall & refund wave. The crucial thing to note is ensuring that errors and defects are spotted as early in the process as possible. The later you notice them, the more time and money they will take to fix. Designing tests that check for quality at every step would be the best way to go; consider error-proofing the process.

Software Development Processes: Team Boards

Software Development Processes: Team Boards



Software Development Processes: Team Boards

Why do we use Team Boards?

Team Boards

- Visualise Work in Progress (WIP)
 - WIP limits are beneficial to reduce WIP
- Help to identify bottlenecks that reduce flow
 - Swarming: whole team focuses on resolving bottlenecks
- Facilitate communication and collaboration
 - Good practice: “I can support you here”
- Support a sense of collective ownership
 - Antipattern: “All my tasks are done”
- Most simple board “To Do - Doing - Done”
- More sophisticated boards use different steps
 - E.g. Scrum style (Push Principle)
“To Do - Coding - Testing - Done”
 - E.g. Kanban style (Pull Principle)
“To Do - Coding - *Coding Done* - Testing - Done”
 - Pull Principle can also be used in Scrum Teams

Software Development Processes: The DoR and DoD

DoR: Definition of Ready

Why is it important to think about whether
a User Story is Ready
to be implemented by the team
Before moving it to “ToDo?”

DoR: Definition of Ready

Problem: Requirements Engineering is an iterative process that involves talking to stakeholders (external of the team) which means that the team have to wait for these conversations to happen

If the Requirements Engineering process is not completed for a User Story and the team is asked to start working on it already, they will

- **produce waste** (aka create something that is not needed) and
- **run into waiting states** (have to stop working on the User Story)

DoR: Definition of Ready

Solution: When planning what work to do next, the team agrees whether a User Story is ready or not, supported by a checklist

Example DoR:

- There's clear business value
- Acceptance criteria are clear and defined
- Quality scenarios are clear and defined
- Dependencies to other teams and systems are identified
- User Story is small enough to be completed within a few days

Source for example: [Scrum Alliance Blog](#)

Further watching: [Janet Gregory and Lisa Crispin on the DoR](#)

DoD: Definition of Done

Why is it important to think about how to
assess
a User Story is done
before it is implemented by the team?

DoD: Definition of Done

Problem: “Works on my machine” <> “It’s nil till you ship it”

Different team members understand their responsibilities differently and might stop working on a User Story prematurely. This results in work that is “nearly done” to become invisible but pile up.

DoD: Definition of Done

Solution: To be sure work that is “nearly done” gets “really done”, the team members hold each other accountable for finishing User Stories, supported by a checklist.

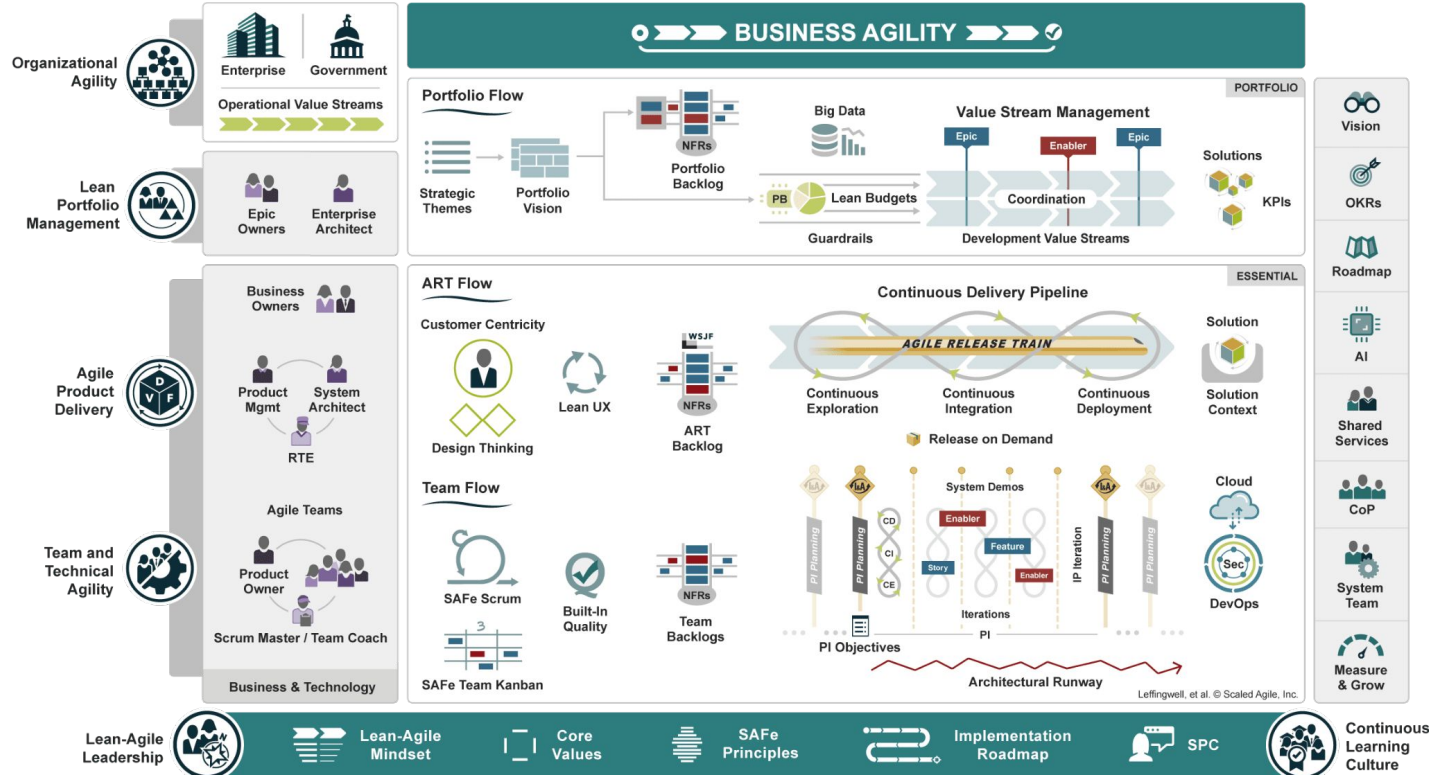
Example DoD:

- Acceptance criteria have been tested and are passing
- Quality scenarios have been tested and are passing
- Automated regression tests have been added
- The change is deployed to production

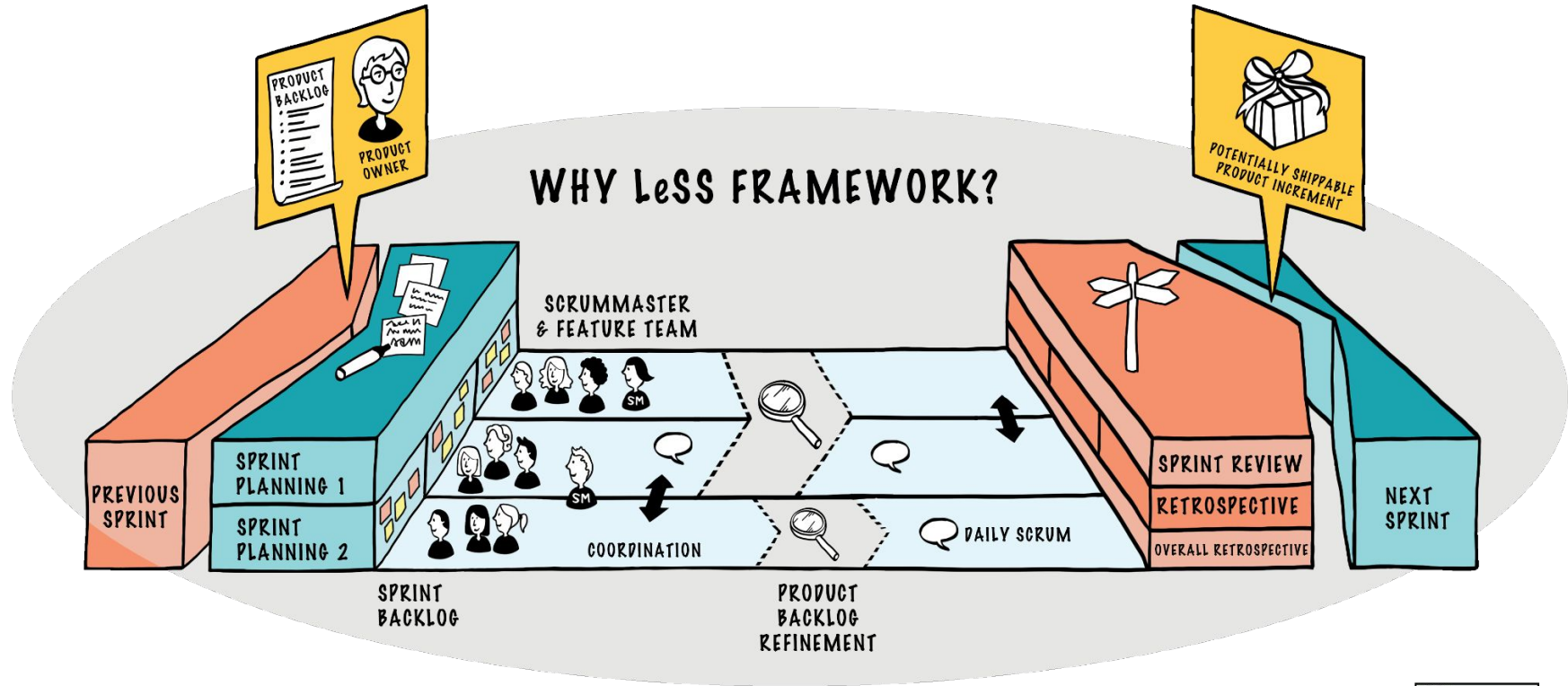
Further watching: [Janet Gregory and Lisa Crispin on the DoD](#)

Software Development Processes: Scaling Agile

Scaled Agile Framework – SAFe – <https://scaledagileframework.com/>



Large Scale Scrum – LeSS – <https://less.works/>

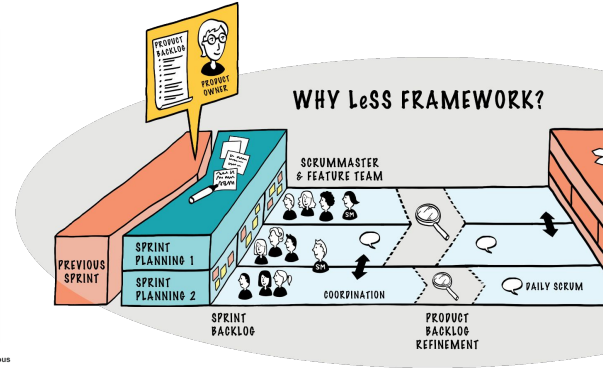
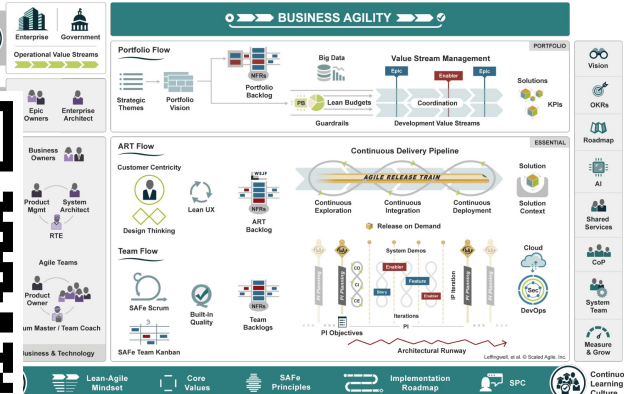


<http://less.works> 

Exercise - 15 minutes

In groups of ~5: Explore the two frameworks to scale agile processes

- What do you discover from the original small scale frameworks?
- What concepts are new?
- What do SAFe and LeSS have in common?
- How do the two frameworks differ?



Further Reading

